BLUEHIRE JOB PORTAL

A PROJECT REPORT

Submitted by

MANPREET SINGH (24BCS12215)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

CU

CHANDIGARH UNIVERSITY

Chandigarh University

April 2026

CERTIFICATE

This is to certify that the project report entitled “BlueHire Job Portal” is a bona fide work carried out by Manpreet Singh (24BCS12215) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering at Chandigarh University during the academic session 2025-2026.

The work presented in this report is based on the design and implementation of a web-based job portal that allows users to register, sign in, browse available job opportunities, apply for roles, remove applications, and review their activity through a dashboard. The project integrates a React frontend, an Express backend, and MongoDB for data persistence.

To the best of our knowledge, this report has not been submitted in part or full to any other university or institution for the award of any degree or diploma.

Project Supervisor	Student Signature

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty of the Department of Computer Science and Engineering, Chandigarh University, for providing the academic environment and guidance required to complete this project successfully.

I am thankful to all teachers, mentors, and lab staff whose suggestions helped in refining the structure, usability, and implementation approach of the BlueHire Job Portal. Their feedback was valuable in improving both the frontend experience and the backend data flow.

I would also like to acknowledge the official documentation of React, Vite, Express, Node.js, and MongoDB, which served as reliable technical references while building and organizing the application.

Finally, I thank my family and peers for their support, encouragement, and constructive discussions throughout the development of this project.

ABSTRACT

BlueHire is a simple full-stack job portal designed to demonstrate the core workflow of a recruitment application. The system enables job seekers to create an account, sign in, explore available job roles, apply for positions, remove submitted applications, and monitor their profile activity through a personalized dashboard.

The frontend of the application is built using React and Vite to provide a responsive single-page experience. The backend is implemented using Express and Node.js, while MongoDB is used for storing users, job listings, and applications. The application also includes seeded sample jobs, secure password hashing using Node.js crypto utilities, CORS protection for development origins, and application-level validation for user actions.

The project focuses on clarity, usability, and a clean separation between interface logic and server-side processing. It is appropriate as an academic mini-project because it demonstrates user authentication, REST API interaction, persistent storage, search functionality, dashboard presentation, and fundamental full-stack architecture in a compact but meaningful way.

TABLE OF CONTENTS

INTRODUCTION

The employment process increasingly depends on digital platforms that connect recruiters and job seekers quickly. Even a basic job portal must solve several important tasks: user registration, session handling, listing management, persistent application tracking, and simple navigation across multiple screens.

BlueHire is developed as a compact demonstration of these tasks. The project provides an accessible interface where users can browse openings, search through job listings, register with an email address, sign in, apply to jobs, and remove applications when required. The design intentionally keeps the feature set focused so that the structure of the full-stack solution remains easy to understand.

From a software engineering perspective, the project is useful because it combines state management on the client side with REST-based communication, database modeling, validation, and basic security handling on the server side. The codebase is small but representative of a real web application pipeline.

- React manages page state, form state, API calls, and client-side filtering.
- Express exposes routes for authentication, jobs, and application management.
- MongoDB stores users, jobs, and submitted applications persistently.
- The system separates frontend rendering from backend business logic.

OBJECTIVES AND SCOPE

The main objectives of the project are as follows:

- To design a simple and user-friendly job portal interface.
- To implement account creation and sign-in for job seekers.
- To store jobs and applications in MongoDB instead of browser-only storage.
- To allow each user to apply for jobs and manage existing applications.
- To demonstrate a complete frontend-backend-database integration workflow.

The scope of the project is limited to job seeker interactions. BlueHire does not currently implement recruiter-side posting, role-based administration, resume upload, interview scheduling, payment workflows, or analytics dashboards. This limited scope is intentional because it allows the application to focus on core CRUD operations, authentication flow, and a maintainable architecture suitable for academic evaluation.

Within this scope, the project successfully demonstrates a realistic mini-portal that can be extended later into a more advanced placement system.

EXISTING SYSTEM AND PROBLEM STATEMENT

Many beginner-level job portal projects rely entirely on local browser storage, use hardcoded authentication, or do not preserve user actions across sessions. Such approaches are acceptable for interface demonstrations, but they do not reflect the architecture of a practical full-stack application.

The main problem addressed by BlueHire is the need for a simple portal where user identity and applications are not lost when the page refreshes or the browser closes. The system therefore introduces persistent storage through MongoDB and formal API routes through Express.

- The application must support a distinct user account instead of anonymous interaction.
- The portal must track which jobs a signed-in user has already applied to.
- The system must prevent duplicate application entries for the same user and job.
- The backend must validate required data and return meaningful responses.

By solving these problems, the project moves beyond a static UI and becomes a structured full-stack web application.

REQUIREMENT ANALYSIS

The requirements for BlueHire can be grouped into functional and non-functional categories.

Type	Requirement
Functional	Allow users to sign up, sign in, browse jobs, search jobs, apply to a job, remove an application, and view applied jobs in a dashboard.
Functional	Seed a starter set of jobs when the database is empty so the portal is immediately usable during demonstration.
Non-functional	Maintain a responsive and easy-to-understand user interface with clear feedback messages for loading, success, and errors.
Non-functional	Persist all important records using MongoDB and expose data through a maintainable REST API using Express.

Software requirements:

- Node.js runtime
- Vite development server
- React and React DOM
- Express, Mongoose, dotenv, and cors
- MongoDB local instance or cloud connection through environment variables

Hardware requirements:

- A standard personal computer or laptop
- Minimum 4 GB RAM for development purposes
- Internet connection if using a hosted MongoDB cluster

SYSTEM ARCHITECTURE

BlueHire follows a three-layer architecture. The presentation layer is handled by the React frontend. The application layer is implemented through an Express server that processes requests, validates inputs, and communicates with the database. The persistence layer is provided by MongoDB through Mongoose models.

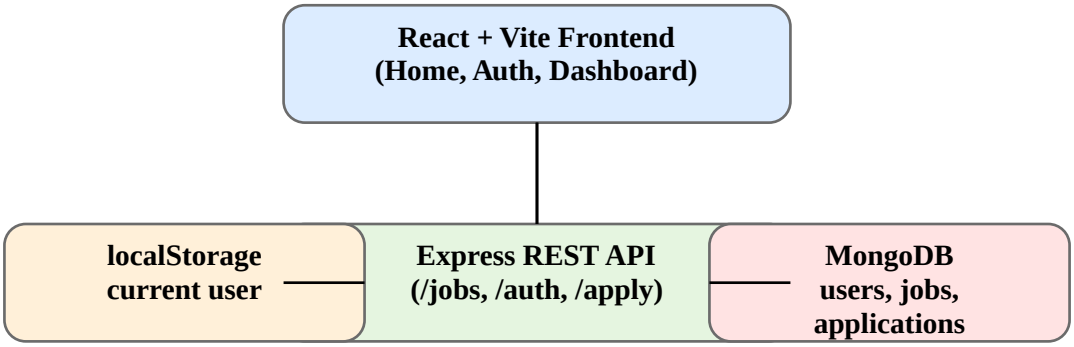

Figure: High-level architecture of the BlueHire Job Portal

The frontend communicates with the backend using fetch calls. The backend identifies the active user by reading the custom X-User-Id request header after sign-in. Applied jobs are stored in a separate Application collection, which references both the user identifier and the selected job.

DATABASE DESIGN

The backend defines three primary schemas: Job, Application, and User. These schemas are intentionally compact but sufficient for a working recruitment workflow.

Collection	Key Fields
User	name, email, passwordHash, passwordSalt, role, location, skills, timestamps
Job	title, company, location, salary, type, skills, timestamps
Application	userId, job reference, status, timestamps, unique index on userId + job

The User collection stores normalized email addresses and derived password data instead of raw passwords. The Job collection holds sample opportunities. The Application collection links a user to a job and stores the application status. A compound unique index ensures that the same user cannot apply to the same job more than once.

API SPECIFICATION

The server exposes a compact REST interface under the /api prefix. Each route has a clear responsibility and returns JSON responses that the React frontend can interpret easily.

Route	Purpose
GET /api/jobs	Returns the list of available jobs sorted by creation order.
GET /api/health	Reports API and database connection status for quick diagnostics.
POST /api/auth/signup	Creates a new user after validating name, email, and password length.
POST /api/auth/signin	Authenticates an existing user and returns public user data.
POST /api/apply	Creates an application for the current user and selected job.
GET /api/applied-jobs	Returns all applied jobs for the signed-in user.
DELETE /api/applications/:jobId	Removes an existing application for the current user.

This API set is sufficient to support the entire visible user flow of the application without introducing unnecessary complexity.

FRONTEND IMPLEMENTATION

The frontend is implemented in React using a single-page structure. The main App component manages page transitions, search input, fetched job data, current user state, loading indicators, feedback messages, and applied job tracking.

- HomePage displays the hero section, search input, status messages, and job cards.
- AuthPage handles sign-up and sign-in modes with shared form logic.
- DashboardPage shows the active user profile and the list of applied jobs.
- JobCard changes its action button based on whether the user has already applied.

The application stores the authenticated user in localStorage under the key bluehireUser. On startup, the stored user is restored safely through a parsing helper. Search is implemented by filtering job title, company, location, and type text on the client side. This keeps the interaction responsive for a small dataset.

BACKEND IMPLEMENTATION

The backend is built with Express and Mongoose. It uses middleware for CORS and JSON parsing, and it loads configuration values through dotenv. If no environment file is provided, the application connects to a local MongoDB instance using sensible defaults.

- Password hashing is implemented with crypto.pbkdf2Sync and a unique per-user salt.
- Password verification uses timingSafeEqual to reduce comparison timing leakage.
- Request validation checks for missing fields, invalid identifiers, and duplicate accounts.
- Job seeding runs automatically when the database starts empty.
- Error handling returns clear JSON messages for both expected and unexpected failures.

The backend also restricts development CORS access to localhost and 127.0.0.1 origins on Vite ports, which keeps the setup practical without opening it broadly during development.

APPLICATION WORKFLOW

The operational workflow of BlueHire is straightforward and can be summarized as follows:

- The server starts, connects to MongoDB, and seeds sample jobs if needed.
- The frontend loads and requests available jobs from the backend API.
- A new user signs up or an existing user signs in through the auth routes.
- The frontend stores the returned public user object in localStorage.
- The signed-in user applies to a job by sending a POST request with the job identifier.
- The dashboard fetches applied jobs through the signed-in user header.
- The user may remove an application using the corresponding DELETE route.

This workflow demonstrates a complete request-response lifecycle, including persistence, retrieval, update of UI state, and controlled deletion.

Because the application separates concerns properly, each stage of the workflow can be debugged and extended independently.

TESTING AND VALIDATION

The project was validated through route-level checks, UI interaction testing, and data persistence verification. Formal automated tests are not included in the current version, but the implemented behaviors can be validated reliably through manual test cases.

Test Case	Expected Result
Create a new account	A valid user is stored in MongoDB and the frontend reports successful account creation.
Sign in with valid credentials	The user is authenticated and the dashboard becomes accessible.
Apply to a job	A new application record is created and the job card state updates to Remove.
Apply twice to the same job	The unique application rule prevents duplicate records.
Remove an application	The selected application is deleted and the dashboard list refreshes.
Open portal without backend	The UI displays an informative backend connection error message.

These tests confirm that the core system responsibilities are working as intended.

RESULTS, STRENGTHS AND LIMITATIONS

The final outcome of the project is a working job portal with persistent account and application management. The major strengths of the system are:

- Clear full-stack separation between UI, API logic, and database storage.
- Persistent applications stored in MongoDB instead of temporary browser-only data.
- Simple and understandable code organization suitable for academic presentation.
- Basic security awareness through password hashing and safe password comparison.

The current limitations are also important to note:

- No recruiter or admin panel is available for posting and managing jobs.
- No JWT or session-token based authentication is implemented yet.
- No resume upload, filtering by category, or recommendation engine is included.
- Automated unit and integration tests are not yet part of the codebase.

CONCLUSION AND FUTURE SCOPE

BlueHire successfully meets the objective of building a compact, database-backed job portal using modern web development tools. The project demonstrates registration, login, listing retrieval, application submission, application removal, dashboard presentation, and persistent storage in a cohesive manner.

As an academic project, it provides a practical example of how frontend and backend systems cooperate to deliver a meaningful user workflow. The implementation is simple enough for explanation yet complete enough to represent a real application pattern.

The future scope of the project includes the following improvements:

- Recruiter dashboard for adding, editing, and deleting job postings
- Role-based authentication and authorization
- Resume upload and profile enrichment
- Search filters based on skills, salary, and location
- Interview scheduling, notifications, and email integration
- Automated tests and deployment configuration

REFERENCES

1. React Documentation. Meta Open Source. https://react.dev/

2. Vite Documentation. https://vite.dev/

3. Express Documentation. https://expressjs.com/

4. Node.js Documentation. https://nodejs.org/

5. MongoDB Manual. https://www.mongodb.com/docs/

6. Mongoose Documentation. https://mongoosejs.com/docs/

7. Project source files and implementation available in the BlueHire codebase submitted with this report.

APPENDIX

Important project files included in the submitted work:

```
BlueHire/
% % %  src/
%    % % %  App.jsx
%    % % %  App.css
%    % % %  main.jsx
% % %  server/
%    % % %  index.js
% % %  README.md
% % %  package.json
% % %  docs/
    % % %  DinosaurGame_ProjectReport.pdf
    % % %  generate_project_report.js
    % % %  BlueHire_ProjectReport_ManpreetSingh_24BCS12215.pdf
```

This appendix shows the compact nature of the project. The application logic is concentrated primarily in one React entry component and one Express server file, which makes the code easy to inspect and discuss during evaluation.